

C Programming - printf() and scanf()

1. The Big Picture

Function	Purpose	Basic syntax	Important idea
printf()	Formatted output	printf("format", values);	Displays text and values.
scanf()	Formatted input	scanf("format", addresses);	Reads values into variables; normally needs their addresses.

```
#include <stdio.h>

int main(void) {
    int age = 20;
    printf("Age: %d\n", age);
    return 0;
}
```

Core rule

The format specifier must match the actual argument type. A format mismatch can cause undefined behavior.

2. printf() - Formatted Output

printf() sends formatted text to standard output. Text inside quotes is printed as written; each conversion specifier beginning with % inserts a value. It returns the number of characters printed, or a negative value if an output error occurs.

Type	Typical printf specifier	Example
int	%d or %i	printf("%d", 42);
unsigned int	%u	printf("%u", 42U);
short	%hd	printf("%hd", x);
long	%ld	printf("%ld", x);
long long	%lld	printf("%lld", x);
float	%f	printf("%f", f);
double	%f	printf("%f", d);

Type	Typical printf specifier	Example
long double	%Lf	printf("%Lf", ld);
char	%c	printf("%c", 'A');
C string (char array)	%s	printf("%s", name);
pointer address	%p	printf("%p", (void *)&x);
hexadecimal integer	%x / %X	printf("%x", 255);
octal integer	%o	printf("%o", 8);
literal percent	%%	printf("50%%");

3. printf() Formatting You Should Know

Format	Meaning	Example output
%.2f	2 digits after decimal	3.14
%8d	Minimum field width 8	42
%-8d	Left-align in width 8	42
%08d	Pad integer with zeroes	00000042
%+d	Always show sign	+42 / -42
%#x	Show hexadecimal prefix	0xff
%e / %E	Scientific notation	1.234000e+03
%g / %G	Compact floating format	1234.5

Format structure

A useful mental model is: % [flags] [width] [.precision] [length] specifier. You do not need to memorize every combination on day one; understand what each part controls.

4. Escape Sequences

Code	Meaning	Example
\n	New line	printf("Hello\nWorld");
\t	Horizontal tab	printf("A\tB");
\\	Backslash	printf("C:\\Temp");
\"	Double quote inside a string	printf("He said \"Hi\"");

5. scanf() - Formatted Input

scanf() reads formatted input from standard input. For ordinary scalar variables, pass the variable address using & because scanf() must write the entered value into that memory location.

```
int age;
printf("Enter age: ");
if (scanf("%d", &age) == 1) {
    printf("You entered %d\n", age);
}
```

Variable type	scanf specifier	Correct argument
int	%d	&number
unsigned int	%u	&number
short	%hd	&number
long	%ld	&number
long long	%lld	&number
float	%f	&number
double	%lf	&number
long double	%Lf	&number
char	%c	&letter
char array / word	%s	name (no &)
hexadecimal int	%x	&number
octal int	%o	&number

The float/double rule

printf(): %f is used for both float and double output because float arguments are promoted to double. scanf(): %f expects float*, while %lf expects double*.

6. %c vs %s - Know the Difference

%c	%s
Reads one character	Reads a string (word)
For char variable: use &	For char array: do not use &
Can read whitespace; " %c" skips leading whitespace	Skips leading whitespace and stops at whitespace
Example: scanf(" %c", &grade);	Example: scanf("%19s", name);

```
char grade;
scanf(" %c", &grade);    // leading space skips leftover whitespace
```

```
char name[20];
scanf("%19s", name);    // max 19 chars + '\0'
```

Why %19s?

If char name[20] is the destination, keep one byte for the terminating null character. A width of 19 limits the input to at most 19 characters.

7. Strings, "\0" and fgets()

A C string is a character array terminated by the null character '\0'. This is why functions that treat data as a string need a valid terminating null character and enough storage.

```
char name[20];
scanf("%19s", name);

// For a full line containing spaces:
char fullName[100];
fgets(fullName, sizeof fullName, stdin);
```

Professional habit

Use fgets() when you need a complete line that may contain spaces. Do not use gets(); it cannot know the destination buffer size and was removed from the C standard.

8. The Classic scanf() Whitespace Problem

```
int age;
char grade;

scanf("%d", &age);
scanf("%c", &grade); // may read the leftover '\n'

// Safer for interactive character input:
scanf(" %c", &grade);
```

What user types	What remains in input	Why %c can surprise you
20 then Enter	newline (\n)	%d reads 20 but leaves the newline. A plain %c can consume it.

9. Reading Multiple Values

```
int day, month, year;
printf("Enter date as dd/mm/yyyy: ");
scanf("%d/%d/%d", &day, &month, &year);
```

Literal characters in the scanf format string participate in matching. The example expects input such as 25/12/2026.

10. scanf() Return Value - Essential for Robust Programs

Return value	Meaning
1, 2, 3, ...	That many conversions succeeded.
0	The next conversion could not match the input. Invalid text may still be waiting in stdin.
EOF	Input ended before a conversion could be completed (commonly -1).

```
int age;

if (scanf("%d", &age) != 1) {
    printf("Invalid input\n");
}
```

Important

Do not assume all requested variables were populated unless scanf() returned the expected count.

11. What Happens with Invalid Input?

```
int age;
if (scanf("%d", &age) != 1) {
    // If input was "abc", the text "abc" is still in stdin.
    // Repeating scanf("%d", &age) immediately will fail again.
}
```

For simple exercises, checking the return value is enough. For robust interactive programs, a common pattern is to discard the rest of the current line after a failed conversion, or read the line with fgets() and validate it with strtol()/strtod().

```
int ch;
while ((ch = getchar()) != '\n' && ch != EOF) {
    // discard remaining input on the current line
}
```

12. Common Mistakes - printf() and scanf()

Mistake	Correct form	Why
scanf("%d", n);	scanf("%d", &n);	Ordinary variables need their address.
scanf("%f", &d);	scanf("%lf", &d);	double input uses %lf.
scanf("%s", &name);	scanf("%19s", name);	A char array is passed without &, and the width protects the buffer.
scanf("%c", &c);	scanf(" %c", &c);	Leading space can skip leftover whitespace for interactive input.

Mistake	Correct form	Why
<code>printf("%d", 3.14);</code>	<code>printf("%f", 3.14);</code>	The conversion specifier must match the argument type.
<code>printf("%f", x)</code> where x is int	Use %d for int	%f expects a double argument in printf().

13. Data Type -> Format Specifier: Build the Connection

A strong programmer does not memorize format codes in isolation. Think: variable type -> memory/value representation -> correct format specifier -> correct pointer for scanf().

Data type	printf()	scanf()	Input argument
char	%c	%c	&ch
int	%d	%d	&x
float	%f	%f	&f
double	%f	%lf	&d
long	%ld	%ld	&x
long long	%lld	%lld	&x
char[] string	%s	%s	name

Do not oversimplify

For printf(), the %f rule for floating-point output is intentionally different from scanf(). This difference is a frequent interview and exam question.

14. Useful Intermediate Concepts

Concept	What to know	Example
%d vs %i in scanf()	%d reads decimal integer; %i can auto-detect base from prefixes.	<code>scanf("%i", &n);</code>
%zu	Print size_t values.	<code>printf("%zu", sizeof(int));</code>
%[...]	Scanset input; useful but use carefully.	<code>scanf("%[A-Z]", s);</code>
Assignment suppression *	Read input without storing it.	<code>scanf("%d/%d/%d", &d, &y);</code>

15. printf() Return Value

```
int n = printf("Hello");
printf("\n%d\n", n);
```

The first printf prints 5 characters, so n becomes 5. Return values are useful when you need to detect output errors or understand library behavior.

16. Practice Examples - From Simple to Tricky

Example 1 - Basic input/output

```
int age;
scanf("%d", &age);
printf("Age = %d\n", age);
```

Why: Read an int and print it.

Example 2 - Decimal calculation

```
int a = 5, b = 2;
printf("%.2f\n", (a + b) / 2.0);
```

Why: Use 2.0 to avoid integer division.

Example 3 - Character after integer

```
int age; char grade;
scanf("%d", &age);
scanf(" %c", &grade);
printf("%d %c\n", age, grade);
```

Why: Leading space before %c handles leftover whitespace.

Example 4 - Multiple values

```
int a, b;
if (scanf("%d %d", &a, &b) == 2)
    printf("Sum = %d\n", a + b);
```

Why: Check that both conversions succeeded.

Example 5 - Safe word input

```
char name[20];
scanf("%19s", name);
printf("Hello %s\n", name);
```

Why: Protect the array from overflow.

Example 6 - Full line

```
char name[100];
fgets(name, sizeof name, stdin);
```

Why: Use for names/sentences with spaces.

17. Quick Debugging Checklist

Symptom	First thing to check
Output looks wrong	Check every printf specifier against the actual argument type.
scanf seems to fail	Check the return value; inspect the input for invalid text.
Character input is skipped	Try " %c" when leading whitespace is not meaningful.

Symptom	First thing to check
String is corrupted / crash	Check array size, null termination, and %s width.
Only part of input is read	Remember %s stops at whitespace; use fgets() for a full line.
Compiler is silent	Compile with warnings such as -Wall -Wextra; format mismatches are often diagnosed.

18. Industry Habits to Build Early

- Match format specifier and actual type exactly.
- Use & for ordinary scanf() variables; do not use & for a char array passed to %s.
- For %s, use a width that leaves room for '\0'.
- Use " %c" for interactive character input when leading whitespace is not significant.
- Check scanf() return values when input can be invalid.
- Prefer fgets() + validation for robust line-based input.
- Never use gets().
- Compile with warnings enabled and treat warnings seriously.
- Keep input/output logic clear enough that another programmer can understand the expected input format.

19. Think Like a Programmer - Quick Checks

Q1 - What happens?

```
int a; scanf("%d", a);
```

Why is this wrong?

Q2 - What happens?

```
float f=2.5; printf("%d", f);
```

Is this a type mismatch?

Q3 - What prints?

```
printf("%d", 7/2);
```

Why is the answer not 3.5?

Q4 - What is the risk?

```
char name[5]; scanf("%s", name);
```

What if the user enters a long word?

Q5 - Why the leading space?

```
scanf(" %c", &ch);
```

What input issue does it prevent?

Q6 - What should be checked?

```
scanf("%d %d", &a, &b);
```

What return value confirms both were read?

20. One-Page Memory Map

Remember	Key idea
printf()	Output; format string + matching values.
scanf()	Input; format string + addresses (except char arrays with %s).
%d	int
%f in printf()	float and double output
%f in scanf()	float*
%lf in scanf()	double*
%c	single character
%s	C string; use width for safety
" %c"	Skip leading whitespace before character input
\n	New line
&x	Address of x
scanf return value	Number of successful conversions; EOF is separate
fgets()	Whole-line input, including spaces
\0	String terminator
-Wall -Wextra	Useful compiler warnings

Final takeaway

Great C programmers do not merely know that %d means int. They understand why scanf needs an address, why %f and %lf differ between printf and scanf, why whitespace affects %c, how strings end with '\0', and how to detect invalid input safely.